



Jerry Lin <jerry@museder.com>

[WordPress Plugin Directory] Review in Progress: Museder RestoreOne

WordPress.org Plugin Directory <plugins@wordpress.org>

2026年4月23日 清晨5:17

收件者: Jerry Lin <jerry@museder.com>

🙌 Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please check this email thoroughly, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

Determine files and directories locations correctly

WordPress provides several functions for easily determining where a given file or directory lives.

We detected that **the way your plugin references some files**, directories and/or URLs **may not work with all WordPress setups**. This happens because there are hardcoded references or you are using the WordPress internal constants.

Let's improve it, please check out the following documentation:

<https://developer.wordpress.org/plugins/plugin-basics/determining-plugin-and-content-directories/>

It contains all the functions available to determine locations correctly.

Most common cases in plugins can be solved using the following functions:

- For where your plugin is located: `plugin_dir_path()`, `plugin_dir_url()`, `plugins_url()`
- For the uploads directory: `wp_upload_dir()` (*Note: If you need to write files, please do so in a folder in the uploads directory, not in your plugin directories*).

Example(s) from your plugin:

```
includes/helpers.php:1249 $core = ABSPATH . 'wp-admin/includes/class-pclzip.php';
includes/class-restore.php:300 $upgrade = rtrim( (string) ABSPATH, '/' ) . '/wp-admin/includes/upgrade.php';
includes/class-backup.php:4521 $prefixes[] = wp_normalize_path( trailingslashit( WP_LANG_DIR ) );
```

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

Complete your checklist.

- ✓ **I fixed all the issues** in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. **I am familiar with tools** such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.
- ✓ **I tested my updated plugin on a clean WordPress installation** with **WP_DEBUG set to true**.

 **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

- ✓ **I acknowledge that this review will be rejected** if I overlook the issues or fail to test my code.
- ✓ **I went to "Add your plugin"** and **uploaded the updated version**. *I can continue updating the code there throughout the review process — the team will always check the latest version.*
- ✓ **I replied to this email**. I was concise and shared any clarifications or important context that the team needed to know.
I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

 To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, **we ask you to thoroughly check all shared issues and fix them before sending the code back to us**. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T12 22Apr26/3.9 (P0TDX263980HGN)

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

[隱藏引用文字]

[隱藏引用文字]

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

[隱藏引用文字]

[隱藏引用文字]

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

[隱藏引用文字]

[隱藏引用文字]

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- `function musere_save_post(){ ... }`
- `class MUSERE_Admin { ... }`
- `update_option( 'musere_options', $options );`
- `add_shortcode( 'musere_shortcode', $callback );`
- `register_setting( 'musere_settings', 'musere_user_id', ... );`
- `define( 'MUSERE_PLUGIN_DIR', plugin_dir_path( __FILE__ ) );`
- `global $musere_options;`
- `add_action('wp_ajax_musere_save_data', ... );`
- `namespace artherslin\musederrestoreone;`

[隱藏引用文字]

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Tue, Mar 24, 2026 at 3:13 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

This is an automated message to confirm that we have received your updated plugin file.

File updated by artherslin, version 2.7.246.

Comment: Dear Plugin Review Team,

Thank you for your continued review of **Museder RestoreOne**. I have uploaded **version 2.7.246**.

This update specifically addresses the WordPress.org review concerns:

- **Guideline 5 / locked functionality:** the free package no longer bundles premium activation, license checks, or separate PRO modules. Optional functionality is only available when a **separate add-on plugin** is installed.
- **Readme disclosure / external services:** the readme now clearly states that the plugin **does not use third-party external services**. It also explains the short, non-blocking **loopback request** to this site's own `wp-cron.php` used to help scheduled tasks run.
- **Readme structure:** the readme now has a **single Changelog** section.
- **REST API permissions:** All routes now use the plugin-prefixed namespace **museder-restoreone/v1**, and the REST endpoints use proper **permission_callback** checks rather than being publicly writable.
- **Security / request handling:** the plugin keeps nonce verification, capability checks, sanitization/validation, direct file access guards, and avoids unsafe direct WordPress bootstrap patterns.

Before upload, I checked the code with **Plugin Check 1.7.0** bundled PHPCS ruleset (`'plugin-review.xml'`), which reported **0 errors and 0 warnings**.

If anything still needs adjustment, I am happy to follow up.

Best regards,

Jerry Lin

Author of Museder RestoreOne

https://tw.wordpress.org/plugins/files/2025/11/24_15-13-03_museder-restoreone.zip

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Tue, Mar 17, 2026 at 4:58 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

👋 Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please **check this email thoroughly**, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

[隱藏引用文字]

[隱藏引用文字]

Example(s) from your plugin:

```
includes/class-restore-handler.php:299 set_time_limit(600);
includes/class-backup.php:4037 set_time_limit(0);
```

Review: Missing permission_callback in REST API Route

When using `register_rest_route()` or `wp_register_ability()` to define custom REST API endpoints, it is crucial to include a proper `permission_callback` .

🔒 This callback function ensures that only authorized users can access or modify data through your endpoint.

Code example, checking that the user can change options:

```
register_rest_route( 'museder-restoreone/v1', '/my-endpoint', array(
    'methods' => 'GET',
    'callback' => 'museder-restoreone_callback_function',
    'permission_callback' => function() {
        return current_user_can( 'manage_options' );
    }
) );
```

Please check the [register_rest_route\(\) documentation](#) and the [current_user_can\(\) documentation](#).

✅ When a permission_callback is NOT Required:

There are valid use cases for public endpoints, such as publicly available data (e.g., posts, public metadata) or endpoints designed for unauthenticated access (e.g., fetching public stats or information).

In these cases, you should use `__return_true` as the `permission_callback` to indicate that the endpoint is intentionally public.

🔒 When a permission_callback IS Required:

For endpoints that involve sensitive data or actions (e.g., getting not public data, creating, updating, or deleting content).

In these cases, you should always implement proper permission checks.

Possible cases found on this plugin's code:

```
includes/pro/ai-controller.php:86 register_rest_route(self::NAMESPACE, '/' . self::BASE . '/nl-command', ['methods' => 'POST', 'callback' => [__CLASS__, 'handle_nl_command'], 'permission_callback' => [__CLASS__, 'check_permission']]);
# ↳ Detected: check_permission
# ✨ check_permission checks manage_options but only requires a non-empty nonce value and does not actually verify it (wp_verify_nonce missing).
includes/pro/ai-controller.php:76 register_rest_route(self::NAMESPACE, '/' . self::BASE . '/smart-schedule', ['methods' => 'GET', 'callback' => [__CLASS__, 'handle_smart_schedule'], 'permission_callback' => [__CLASS__, 'check_permission']]);
# ↳ Detected: check_permission
# ✨ check_permission checks manage_options but only requires a non-empty nonce value and does not actually verify it (wp_verify_nonce missing).
```

[隱藏引用文字]

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', \$options);
- add_shortcode('musere_shortcode', \$callback);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global \$musere_options;
- add_action('wp_ajax_musere_save_data', ...);
- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if (!function_exists('NAME')) {` around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using `if-exists` should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

```
# This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2",  
"musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonerestore",  
"musederrestoreonereports", "musederrestoreoneadminui" for 167 element(s).
```

```
# Looks like there are elements not using common prefixes.  
includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');  
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');
```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');  
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');
```

👉 **Continue with the review process.**

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 **Complete your checklist.**

✓ **I fixed all the issues** in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. **I am familiar with tools** such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.

✓ I tested my updated plugin on a clean WordPress installation with **WP_DEBUG** set to true.

⚠ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.

✓ I went to **"Add your plugin"** and uploaded the updated version. *I can continue updating the code there throughout the review process — the team will always check the latest version.*

✓ I replied to this email. I was concise and shared any clarifications or important context that the team needed to know.

I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

i To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, **we ask you to thoroughly check all shared issues and fix them before sending the code back to us.** *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder–restoreone/artherslin/1Dec25/T9 17Mar26/3.8

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Tue, Mar 10, 2026 at 6:18 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org>

wrote:

This is an automated message to confirm that we have received your updated plugin file.

File updated by artherslin, version 2.7.243.

Comment: ****Subject:**** Museder RestoreOne – Updated version 2.7.243 submitted for review

Dear Plugin Review Team,

Thank you for your previous feedback on Museder RestoreOne. I have addressed the issues raised and am submitting ****version 2.7.243**** for review.

****Summary of compliance in this version:****

- ****Plugin URI**** is reachable: <https://musederlabs.com/restoreone-plugin/>
- ****External services**** are documented in the readme (== External services ==): Amazon S3 and OpenAI, with purpose, data sent, when, and links to Terms of Service and Privacy Policy. The plugin does not use external services by default.
- ****Security:**** All relevant handlers use nonce verification and capability checks; input is sanitized/validated and output is escaped. Direct file access is guarded with ``if ( ! defined( 'ABSPATH' ) ) exit;`` where appropriate.
- ****Core loading:**** No direct inclusion of wp-load.php, wp-config.php, or wp-blog-header.php. Required core includes (e.g. wp-admin/includes/file.php) are loaded only inside functions when needed.
- ****Paths:**** Plugin uses ``plugin_dir_path()``, ``plugin_dir_url()``, and ``wp_upload_dir()`` for paths and storage; user-generated files are stored under the uploads directory.
- ****PHP limits:**** ``set_time_limit()`` is used only inside long-running functions (backup/restore), not in init or global scope.
- ****Input handling:**** The plugin no longer processes the whole request body via ``file_get_contents('php://input')``; only the necessary fields are read and validated.
- ****Options/transients:**** Option names use a clear, statically visible prefix (e.g. ``museder_restoreone_job_lock_`` via a constant).
- ****Plugin Check:**** The code has been checked with the Plugin Check plugin's PHPCS ruleset (plugin-review.xml), resulting in ****0 errors and 0 warnings****.

I have tested the updated plugin on a clean WordPress installation with WP_DEBUG enabled. The latest code has been uploaded to the plugin directory for your review.

Thank you for your time. I am happy to provide any further information or clarification you need.

Best regards,
Jerry Lin
Author of Museder RestoreOne

https://wordpress.org/plugins/files/2025/11/10_18-18-53_museder-restoreone.zip

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

[隱藏引用文字]

[隱藏引用文字]

Example(s) from your plugin:

```
includes/class-restore-handler.php:509 require_once ABSPATH . 'wp-admin/includes/media.php';
```

[隱藏引用文字]

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', \$options);
- add_shortcode('musere_shortcode', \$callback);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global \$musere_options;
- add_action('wp_ajax_musere_save_data', ...);
- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of __ (double underscores), wp_ , or _ (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using _n() or __() for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using if (!function_exists('NAME')) { around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using if-exists should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

```
# This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2",  
"musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonerestore",  
"musederrestoreonereports", "musederrestoreoneadminui" for 169 element(s).
```

```
# Looks like there are elements not using common prefixes.
```

```
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
```

```
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

```
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
```

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 Complete your checklist.

✓ **I fixed all the issues** in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. **I am familiar with tools** such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.

✓ I tested my updated plugin on a clean WordPress installation with `WP_DEBUG` set to `true`.

⚠ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.

✓ I went to "Add your plugin" and uploaded the updated version. *I can continue updating the code there throughout the review process — the team will always check the latest version.*

✓ I replied to **this** email. I was concise and shared any clarifications or important context that the team needed to know.

I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

i To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, **we ask you to thoroughly check all shared issues and fix them before sending the code back to us.** *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T8 7Feb26/3.8RC3

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

[隱藏引用文字]

[隱藏引用文字]

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Fri, Jan 23, 2026 at 7:10 PM UTC, WordPress.org Plugin Directory

<plugins@wordpress.org> wrote:



Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please **check this email thoroughly**, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

AI generated output.

We've detected files that appear to be the output of an AI tool used during development.

These files usually include explanations or summaries of the changes made by the AI. That's great! We appreciate the effort you're putting into improving your plugin. However, those files aren't necessary for the plugin functionality.

They are also typically not useful as documentation because they tend to be partial and unconnected files that are also not maintained or updated.

We encourage you to use the available tools to improve your plugin. However, please remove any unnecessary AI-generated output. This will help keep the plugin package clean and reduce potential confusion for users and reviewers.

From your plugin:

museder-restoreone/docs/wp-compliance-checklist.md

Generic function/class/define/namespace/option names

All plugins must have unique function names, namespaces, defines, class and option names. This prevents your plugin from conflicting with other plugins or themes. We need you to update your plugin to use more unique and distinct names.

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', \$options);
- add_shortcode('musere_shortcode', \$callback);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global \$musere_options;
- add_action('wp_ajax_musere_save_data', ...);
- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if (!function_exists('NAME')) {` around all your functions and classes

sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using if-exists should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

```
# This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2",  
"musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonereports",  
"musederrestoreonerestore", "musederrestoreoneadminui" for 22 element(s).  
# This plugin is using the prefix "backup_lite" for 146 element(s).
```

```
# Looks like there are elements not using common prefixes.  
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');  
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');  
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

Allowing direct file access to plugin files

Direct file access occurs when someone directly queries a PHP file. This can be done by entering the complete path to the file in the browser's URL bar or by sending a POST request directly to the file.

For files that only contain class or function definitions, the risk of something funky happening when accessed directly is minimal. However, for files that contain executable code (e.g., function calls, class instance creation, class method calls, or inclusion of other PHP files), the risk of security issues is hard to predict because it depends on the specific case, but it can exist and it can be high.

You can easily prevent this by adding the following code at the beginning of all PHP files that could potentially execute code if accessed directly:

```
if ( ! defined( 'ABSPATH' ) ) exit; // Exit if accessed directly
```

Add it after the <?php opening tag and after the namespace declaration, if any, but before any other code.

Example(s) from your plugin:

```
download-handler.php:19
```

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 Complete your checklist.

✓ **I fixed all the issues** in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. **I am familiar with tools** such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.

✓ **I tested my updated plugin on a clean WordPress installation** with [WP_DEBUG set to true](#).

⚠️ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

✓ **I acknowledge that this review will be rejected** if I overlook the issues or fail to test my code.

✓ **I went to "Add your plugin"** and **uploaded the updated version**. *I can continue updating the code there throughout the review process — the team will always check the latest version.*

✓ I replied to **this** email. I was concise and shared any clarifications or important context that the team needed to know.

I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

📄 To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, **we ask you to thoroughly check all shared issues and fix them before sending the code back to us**. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T7 23Jan26/3.8RC2

[隱藏引用文字]